OPTIMIZATION WITH SIMD (ARM NEON)

2026 SPRING

JINHO LEE

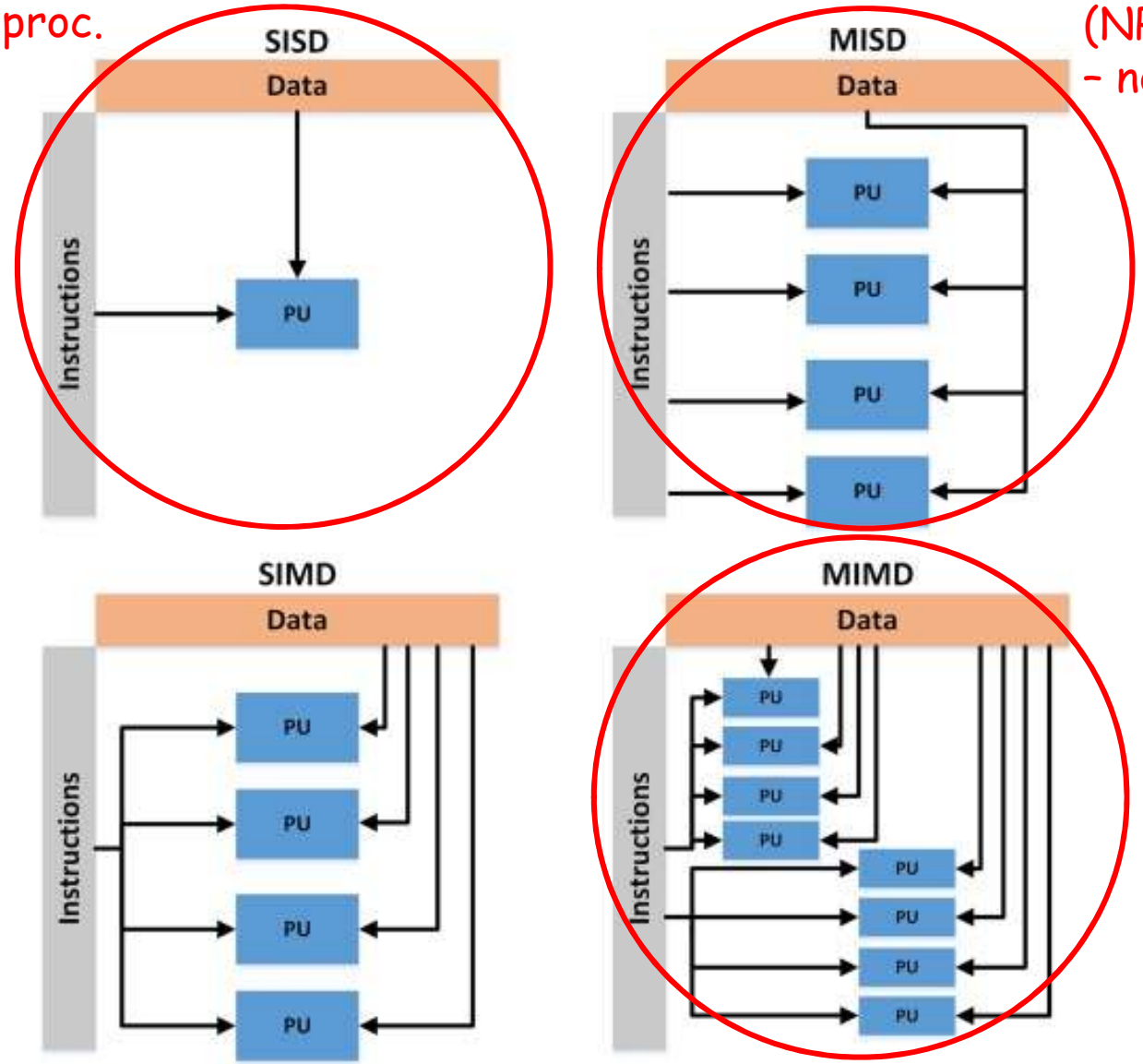
ECE, SNU

Accelerated Intelligent Systems Lab.

FLYNN'S TAXONOMY

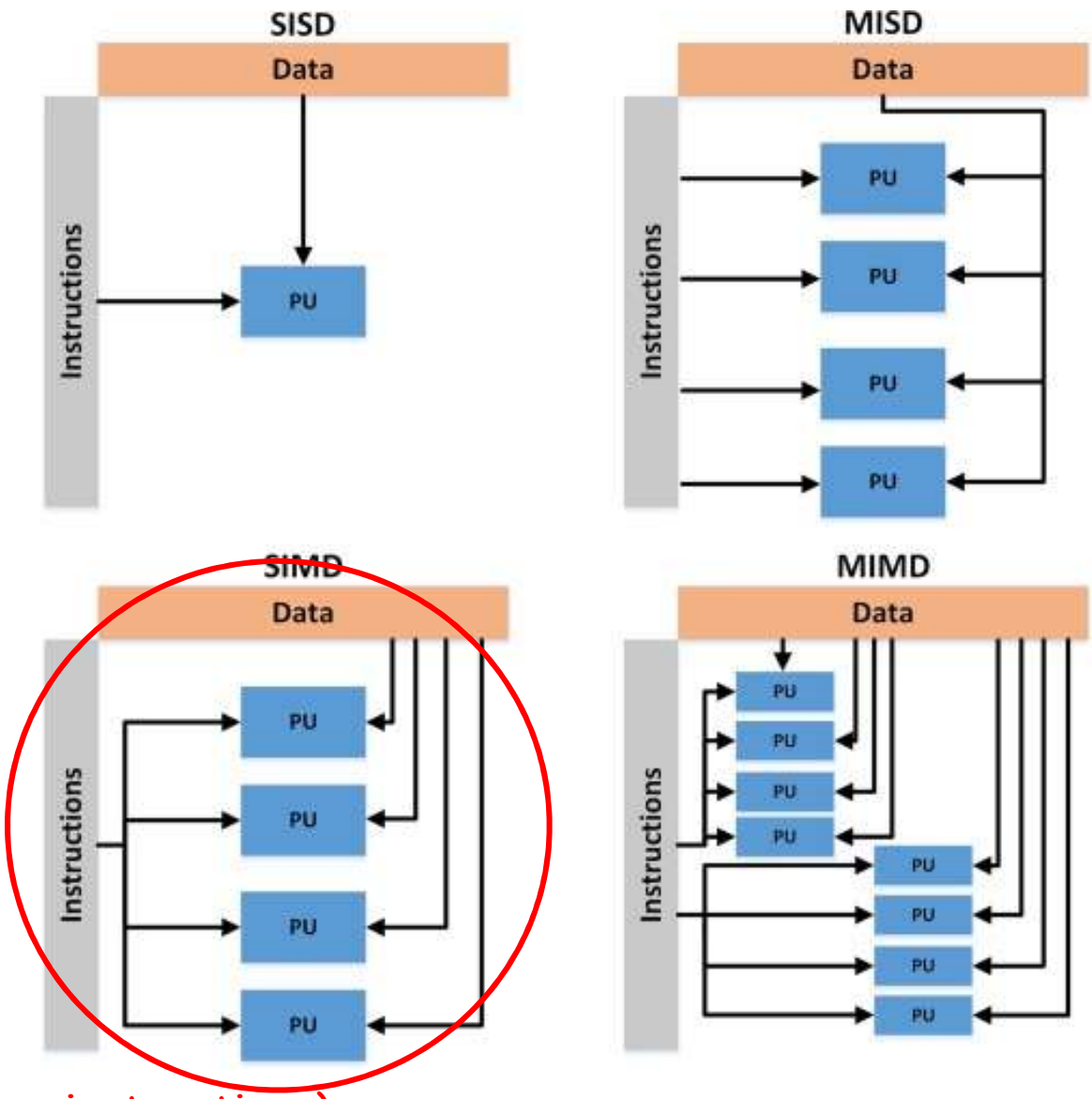
single-core proc.

Systolic array
(NPU, CGRA)
– not for this class



multi-core proc.

FLYNN'S TAXONOMY

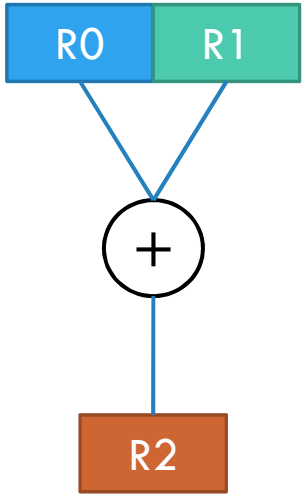


SIMD (Vector instructions)

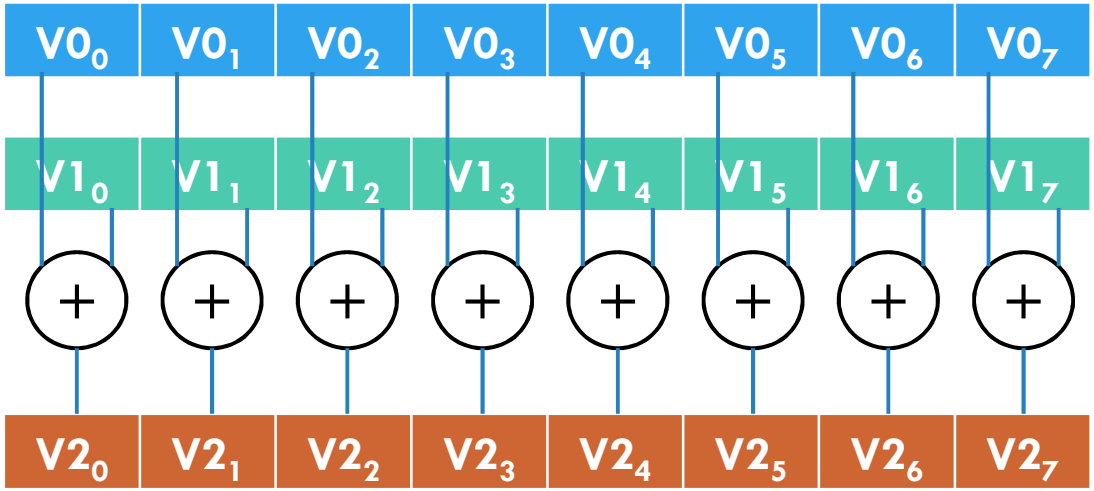
SISD/SIMD INSTRUCTIONS

- One instruction, one data
- One instruction, many data

- `add R2, R0, R1`

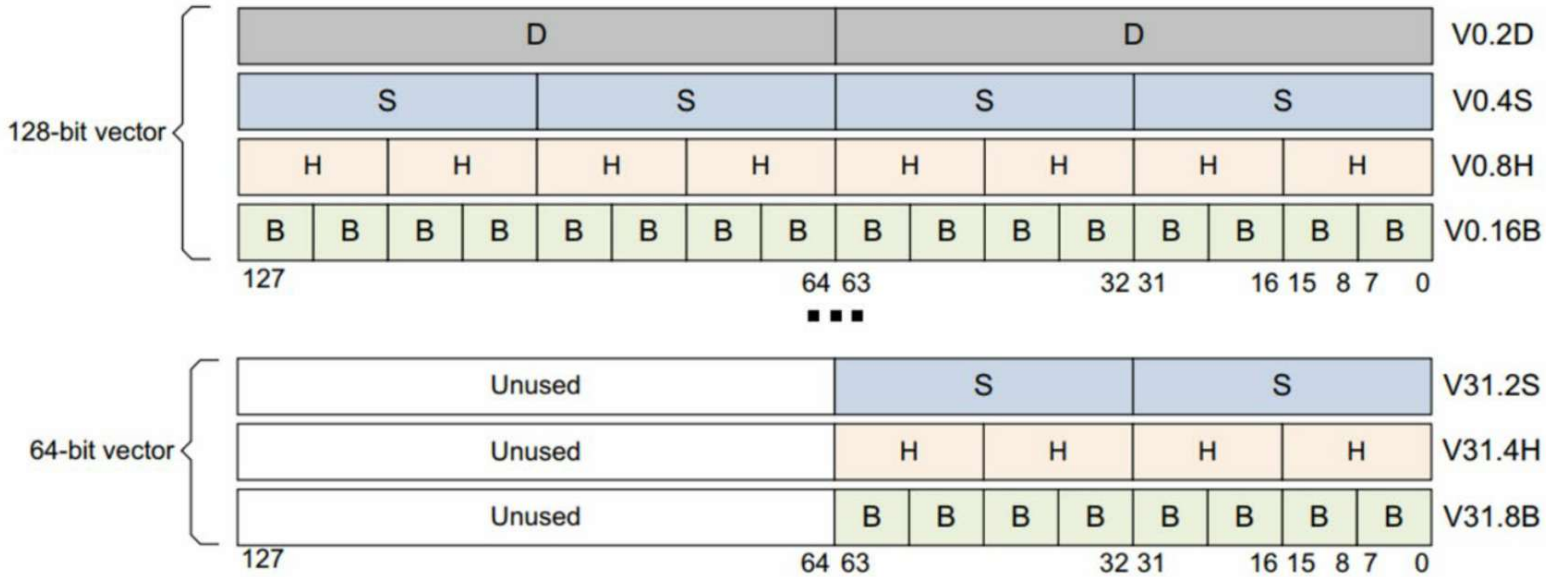


- `vaddq_u32 V2, V0, V1`
  - 8 adds with one instruction



SIMD REGISTER

- SIMD instructions run out of SIMD registers
- Arm Neon has 128-bit registers, which can be used as int/float with
  - 2 * 64b doublewords (D)
  - 4 * 32b singlewords (S)
  - 8 * 16b halfwords (H)
  - 16 * 8b bytes (B)
- Enough arithmetic units are placed in the processor



'Q' registers

'D' registers

COMPUTING PROCEDURE WITH SIMD

- To use SIMD instructions, data must be put on the SIMD registers first
- Thus, the typical pattern is

1. Load data to the SIMD registers (usually from the memory)
2. Compute
3. Store data from SIMD registers (usually to the memory)

PROGRAMMING WITH SIMD INSTRUCTIONS

1. Rely on compilers
   - In theory, compiler should be able to use Neon without any user effort
   - In practice, compilers are not so good at it
   - A bit of hand-optimization is usually beneficial
2. Assembly
   - Assembly (or inline assembly) gives you almost a full control for the maximum performance
   - But nobody like to code assembly in 21st century..
3. <u>Intrinsics</u>
   - Built-in functions corresponding to the SIMD instructions
   - Strong suggestion to place SIMD instructions in the compilation result
   - Easier than assembly, almost all the performance you can get from assembly

VECTOR ADD WITH SIMD

- `#include <arm_neon.h>`

```
void vadd_serial(uint32_t* inA, uint32_t* inB, uint32_t* outC, uint32_t len)
{
    for(uint32_t i=0;i<len;i++) {
        outC[i] = inA[i] + inB[i];
    }
}
```

```
#define SIMD_LEN 4
void vadd_serial_simd(uint32_t* inA, uint32_t* inB, uint32_t* outC, uint32_t len)
{
    for(uint32_t i=0;i<len;i+=SIMD_LEN) {
        uint32x4_t A_reg = vld1q_u32(inA + i);
        uint32x4_t B_reg = vld1q_u32(inB + i);
        uint32x4_t C_reg = vaddq_u32(A_reg, B_reg);
        vst1q_u32(outC + i, C_reg); }
}
```

VECTOR ADD WITH SIMD

- `#include <arm_neon.h>`

```
void vadd_serial(uint32_t* inA, uint32_t* inB, uint32_t* outC, uint32_t len)
{
    for(uint32_t i=0;i<len;i++) {
        outC[i] = inA[i] + inB[i];
    }
}
```

```
#define SIMD_LEN 4
void vadd_serial_simd(uint32_t* inA, uint32_t* inB, uint32_t* outC, uint32_t len)
{
    for(uint32_t i=0;i<len;i+=SIMD_LEN) {
        uint32x4_t A_reg = vld1q_u32(inA + i);
        uint32x4_t B_reg = vld1q_u32(inB + i);
        uint32x4_t C_reg = vaddq_u32(A_reg, B_reg);
        vst1q_u32(outC + i, C_reg);
    }
}
```

Load

Compute

Store

VECTOR ADD WITH SIMD

- `#include <arm_neon.h>`

```
void vadd_serial(uint32_t* inA, uint32_t* inB, uint32_t* outC, uint32_t len)
{
    for(uint32_t i=0;i<len;i++) {
        outC[i] = inA[i] + inB[i];
    }
}
```

we're going to work on 32b ints, so we have 4-way simd (128 = 32 x 4)

```
#define SIMD_LEN 4
void vadd_serial_simd(uint32_t* inA, uint32_t* inB, uint32_t* outC, uint32_t len)
{
    for(uint32_t i=0;i<len;i+=SIMD_LEN) {
        uint32x4_t A_reg = vld1q_u32(inA + i);
        uint32x4_t B_reg = vld1q_u32(inB + i);
        uint32x4_t C_reg = vaddq_u32(A_reg, B_reg);
        vst1q_u32(outC + i, C_reg);
    }
}
```

VECTOR ADD WITH SIMD

- `#include <arm_neon.h>`

```
void vadd_serial(uint32_t* inA, uint32_t* inB, uint32_t* outC, uint32_t len)
{
    for(uint32_t i=0;i<len;i++) {
        outC[i] = inA[i] + inB[i];
    }
}
```

```
#define SIMD_LEN 4
void vadd_serial_simd(uint32_t* inA, uint32_t* inB, uint32_t* outC, uint32_t len)
{
    for(uint32_t i=0;i<len;i+=SIMD_LEN) {
        uint32x4_t A_reg = vld1q_u32(inA + i);
        uint32x4_t B_reg = vld1q_u32(inB + i);
        uint32x4_t C_reg = vaddq_u32(A_reg, B_reg);
        vst1q_u32(outC + i, C_reg);
    }
}
```

- neon vector types.
- `<type><size>x<number_of_lanes>_t`
- unsigned int, 32b, 4 lanes

VECTOR ADD WITH SIMD

- `#include <arm_neon.h>`

```
void vadd_serial(uint32_t* inA, uint32_t* inB, uint32_t* outC, uint32_t len)
{
    for(uint32_t i=0;i<len;i++) {
        outC[i] = inA[i] + inB[i];
    }
}
```

```
#define SIMD_LEN 4
void vadd_serial_simd(uint32_t* inA, uint32_t* inB, uint32_t* outC, uint32_t len)
{
    for(uint32_t i=0;i<len;i+=SIMD_LEN) {
        uint32x4_t A_reg = vld1q_u32(inA + i);
        uint32x4_t B_reg = vld1q_u32(inB + i);
        uint32x4_t C_reg = vadd_u32(A_reg, B_reg);
        vst1q_u32(outC + i, C_reg);
    }
}
```

- `vld1q_u32`
- `v ld 1 q _ u32`

vector

load

stride 1

use ‘Q’ register (128b)

unsigned 32b operations

VECTOR ADD WITH SIMD

- `#include <arm_neon.h>`

```
void vadd_serial(uint32_t* inA, uint32_t* inB, uint32_t* outC, uint32_t len)
{
    for(uint32_t i=0;i<len;i++) {
        outC[i] = inA[i] + inB[i];
    }
}
```

```
#define SIMD_LEN 4
void vadd_serial_simd(uint32_t* inA, uint32_t* inB, uint32_t* outC, uint32_t len)
{
    for(uint32_t i=0;i<len;i+=SIMD_LEN) {
        uint32x4_t A_reg = vld1q_u32(inA + i);
        uint32x4_t B_reg = vld1q_u32(inB + i);
        uint32x4_t C_reg = vaddq_u32(A_reg, B_reg);
        vst1q_u32(outC + i, C_reg);
    }
}
```

- add the four 32b uints in parallel $C \leftarrow A + B$
- note the 'q' suffix again for using the Q registers (128b)

VECTOR ADD WITH SIMD

- `#include <arm_neon.h>`

```
void vadd_serial(uint32_t* inA, uint32_t* inB, uint32_t* outC, uint32_t len)
{
    for(uint32_t i=0;i<len;i++) {
        outC[i] = inA[i] + inB[i];
    }
}
```

```
#define SIMD_LEN 4
void vadd_serial_simd(uint32_t* inA, uint32_t* inB, uint32_t* outC, uint32_t len)
{
    for(uint32_t i=0;i<len;i+=SIMD_LEN) {
        uint32x4_t A_reg = vld1q_u32(inA + i);
        uint32x4_t B_reg = vld1q_u32(inB + i);
        uint32x4_t C_reg = vaddq_u32(A_reg, B_reg);
        vst1q_u32(outC + i, C_reg);
    }
}
```

- vector store stride1 Qreg uint32
- into address outC [i] from C_reg

NEON VECTOR TYPES

- All sorts of datatypes exist that fit into D or Q registers
- All with associated computation instructions (intrinsics)
  - e.g., add, sub, mul, etc

64-bit type (D-register)	128-bit type (Q-register)
int8x8_t	int8x16_t
int16x4_t	int16x8_t
int32x2_t	int32x4_t
int64x1_t	int64x2_t
uint8x8_t	uint8x16_t
uint16x4_t	uint16x8_t
uint32x2_t	uint32x4_t
uint64x1_t	uint64x2_t
float16x4_t	float16x8_t
float32x2_t	float32x4_t
poly8x8_t	poly8x16_t
poly16x4_t	poly16x8_t

VECTOR ADD (FP32 VERSION)

- #include <arm_neon.h>

```
void vadd_serial(float32_t* inA, float32_t* inB, float32_t* outC, uint32_t len)
{
    for(uint32_t i=0;i<len;i++) {
        outC[i] = inA[i] + inB[i];
    }
}
```

```
#define SIMD_LEN 4
void vadd_parallel_simd(float32_t* inA, float32_t* inB, float32_t* outC, uint32_t len)
{
#pragma omp parallel for
    for(uint32_t i=0;i<len;i+=SIMD_LEN) {
        float32x4_t A_reg = vld1q_f32(inA + i);
        float32x4_t B_reg = vld1q_f32(inB + i);
        float32x4_t C_reg = vaddq_f32(A_reg, B_reg);
        vst1q_f32(outC + i, C_reg); }
}
```

- f32 types instead of u32

- float vector variables

STRIDES

- #include <arm_neon.h>

```
void vadd_serial(uint32_t* inA, uint32_t* inB, uint32_t* outC, uint32_t len)
{
    for(uint32_t i=0;i<len;i++) {
        outC[i] = inA[i] + inB[i];
    }
}
```

```
#define SIMD_LEN 4
void vadd_serial_simd(uint32_t* inA, uint32_t* inB, uint32_t* outC, uint32_t len)
{
    for(uint32_t i=0;i<len;i+=SIMD_LEN) {
        uint32x4_t A_reg = vld1q_u32(inA + i);
        uint32x4_t B_reg = vld1q_u32(inB + i);
        uint32x4_t C_reg = vaddq_u32(A_reg, B_reg);
        vst1q_u32(outC + i, C_reg);
    }
}
```

- vld1q_u32
- v ld 1 q _ u32

vector

load

stride 1

use 'Q' register (128b)

unsigned 32b operations

STRIDES

- vld1q_u32
- v ld 1 q _ u32

vector

load

stride 1

use 'Q' register (128b)

unsigned 32b operations

- strides of 1-4 are available for various memory layouts
- E.g., RGB data
  - array-of-structures

R0	G0	B0	R1	G1	B1	R2	..	B3

  - structure-of-arrays

R0	R1	R2	R3	G0	G1	G2	G3	B0

STRIDES

- E.g., RGB data
  - array-of-structures

R0	G0	B0	R1	G1	B1	R2	..	B3

  - structure-of-arrays

R0	R1	R2	R3	G0	G1	G2	G3	B0

- No format is golden, and you will need conversion
  - Between memory and memory
  - Between SIMD register and memory

STRIDES

- array-of-structures

R0	G0	B0	R1	G1	B1	R2	..	B3

- structure-of-arrays

R0	R1	R2	R3	G0	G1	G2	G3	B0

- No format is golden, and you will need conversion

```
void rgb_deinterleave_c(uint8_t *r, uint8_t *g, uint8_t *b, uint8_t *rgb, int len_color) {
    for (int i=0; i < len_color; i++) {
        r[i] = rgb[3*i];
        g[i] = rgb[3*i+1];
        b[i] = rgb[3*i+2];
    }
}
```

```
void rgb_deinterleave_neon(uint8_t *r, uint8_t *g, uint8_t *b, uint8_t *rgb, int len_color) {
int num_regs = len_color / 16;
    uint8x16x3_t intlv_rgb;
    for (int i=0; i < num_regs; i++) {
        intlv_rgb = vld3q_u8(rgb+3*16*i);
        vst1q_u8(r+16*i, intlv_rgb.val[0]);
        vst1q_u8(g+16*i, intlv_rgb.val[1]);
        vst1q_u8(b+16*i, intlv_rgb.val[2]);
    }
}
```

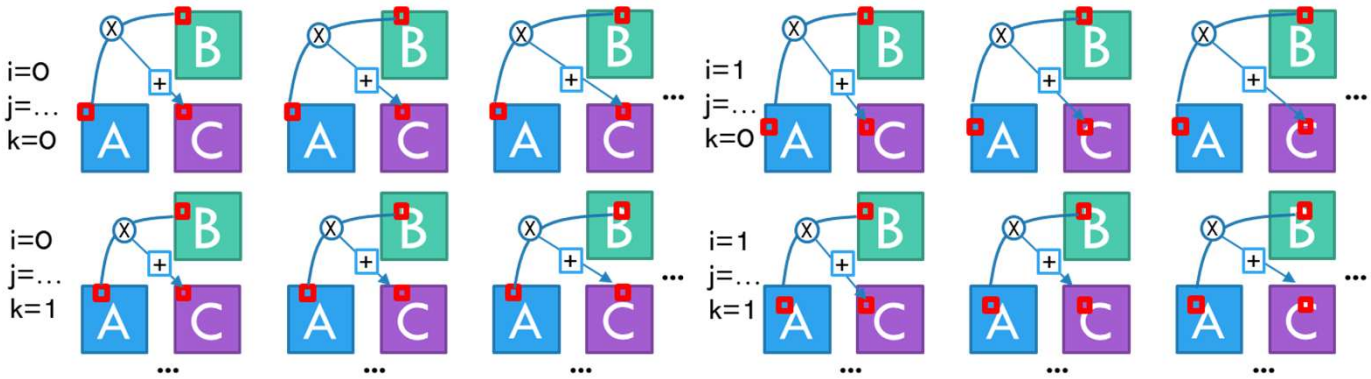
load with stride 3 → array of vectors

```
struct uint8x16x3_t
{
    uint8x4_t val[3];
}
```

USING NEON TO ACCELERATE MATMUL

- SIMD is a type of parallelization, in a slightly restricted form
- and we have beaten matmul parallelization to death so far

- Applying Neon on top is slightly tricky, but not that difficult ☺



METHOD 1: MULTIPLY AND REDUCE

- With 32b elements, we can multiply 4 elements at a time using Neon

$$\begin{bmatrix} x0 & x4 & x8 & xC \\ x1 & x5 & x9 & xD \\ x2 & x6 & xA & xE \\ x3 & x7 & xB & xF \end{bmatrix} \bullet \begin{bmatrix} y0 & y4 & y8 & yC \\ y1 & y5 & y9 & yD \\ y2 & y6 & yA & yE \\ y3 & y7 & yB & yF \end{bmatrix} = \begin{bmatrix} x0y0 + x4y1 + x8y2 + xCy3 & \dots & \dots & \dots \\ x1y0 + x5y1 + x9y2 + xDy3 & \dots & \dots & \dots \\ x2y0 + x6y1 + xAy2 + xEy3 & \dots & \dots & \dots \\ x3y0 + x7y1 + xBy2 + xFy3 & \dots & \dots & \dots \end{bmatrix}$$

- → Multiply four elements from a small subblock
  - We have already talked about blocking, right?

- Performing 4 multiplications
  - `vecC = vmulq_u32(vecA, vecB)`
- Reduce over the results
  - `C[i][j] = vaddvq_u32(vecC)`
  - Warning: horizontal reduction is often slow on SIMD (three additions with dependencies)

METHOD 1: MULTIPLY AND REDUCE

- With 32b elements, we can multiply 4 elements at a time using Neon

Horizontal reduction

$$\begin{bmatrix} x0 & x4 & x8 & xC \\ x1 & x5 & x9 & xD \\ x2 & x6 & xA & xE \\ x3 & x7 & xB & xF \end{bmatrix} \bullet \begin{bmatrix} y0 & y4 & y8 & yC \\ y1 & y5 & y9 & yD \\ y2 & y6 & yA & yE \\ y3 & y7 & yB & yF \end{bmatrix} = \begin{bmatrix} x0y0 + x4y1 + x8y2 + xCy3 & \dots & \dots & \dots \\ x1y0 + x5y1 + x9y2 + xDy3 & \dots & \dots & \dots \\ x2y0 + x6y1 + xAy2 + xEy3 & \dots & \dots & \dots \\ x3y0 + x7y1 + xBy2 + xFy3 & \dots & \dots & \dots \end{bmatrix}$$

- → Multiply four elements from a small subblock
  - We have already talked about blocking, right?

- Performing 4 multiplications
  - `vecC = vmulq_u32(vecA, vecB)`
- Reduce over the results
  - `C[i][j] = vaddvq_u32(vecC)`
  - Warning: horizontal reduction is often slow on SIMD (three additions with dependencies)

METHOD 2: MULTIPLY-ACCUMULATE

- We can alter the algorithm to avoid the reduction

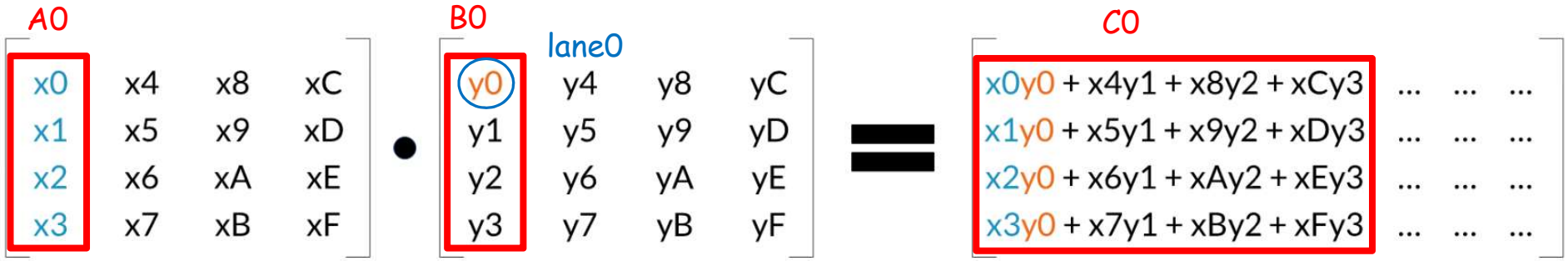
$$\begin{bmatrix} x0 & x4 & x8 & xC \\ x1 & x5 & x9 & xD \\ x2 & x6 & xA & xE \\ x3 & x7 & xB & xF \end{bmatrix} \bullet \begin{bmatrix} y0 & y4 & y8 & yC \\ y1 & y5 & y9 & yD \\ y2 & y6 & yA & yE \\ y3 & y7 & yB & yF \end{bmatrix} = \begin{bmatrix} x0y0 + x4y1 + x8y2 + xCy3 & \dots & \dots & \dots \\ x1y0 + x5y1 + x9y2 + xDy3 & \dots & \dots & \dots \\ x2y0 + x6y1 + xAy2 + xEy3 & \dots & \dots & \dots \\ x3y0 + x7y1 + xBy2 + xFy3 & \dots & \dots & \dots \end{bmatrix}$$

- Elementwise multiplication, followed by elementwise addition (accumulation)
- multiply-accumulate instructions
- e.g., vmlaq_laneq_s32 (a, b, v, lane)

```
 C0 = vmlaq_laneq_s32(C0, A_0, B_0, 0);
 C0 = vmlaq_laneq_s32(C0, A_1, B_0, 1);
 C0 = vmlaq_laneq_s32(C0, A_2, B_0, 2);
 C0 = vmlaq_laneq_s32(C0, A_3, B_0, 3);
...
```

METHOD 2: MULTIPLY-ACCUMULATE

- We can alter the algorithm to avoid the reduction

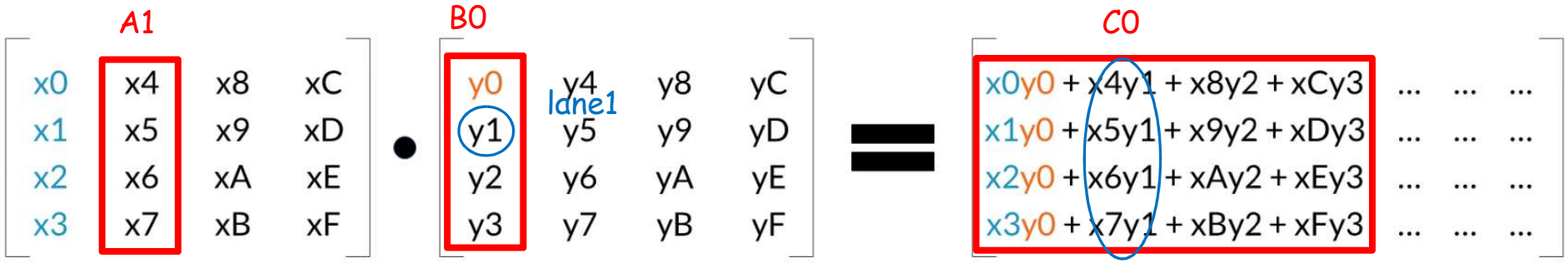


- Elementwise multiplication, followed by elementwise addition (accumulation)
- multiply-accumulate instructions
- e.g., vmlaq_laneq_s32 (a, b, v, lane)

```
 C0 = vmlaq_laneq_s32(C0, A_0, B_0, 0);
 C0 = vmlaq_laneq_s32(C0, A_1, B_0, 1);
 C0 = vmlaq_laneq_s32(C0, A_2, B_0, 2);
 C0 = vmlaq_laneq_s32(C0, A_3, B_0, 3);
...
```

METHOD 2: MULTIPLY-ACCUMULATE

- We can alter the algorithm to avoid the reduction

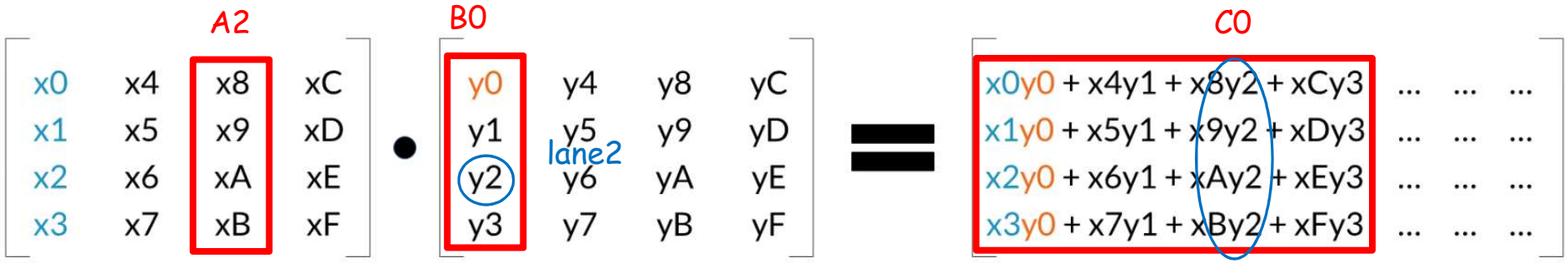


- Elementwise multiplication, followed by elementwise addition (accumulation)
- multiply-accumulate instructions
- e.g., vmlaq_laneq_s32 (a, b, v, lane)

```
 C0 = vmlaq_laneq_s32(C0, A_0, B_0, 0);
 C0 = vmlaq_laneq_s32(C0, A_1, B_0, 1);
 C0 = vmlaq_laneq_s32(C0, A_2, B_0, 2);
 C0 = vmlaq_laneq_s32(C0, A_3, B_0, 3);
...
```

METHOD 2: MULTIPLY-ACCUMULATE

- We can alter the algorithm to avoid the reduction

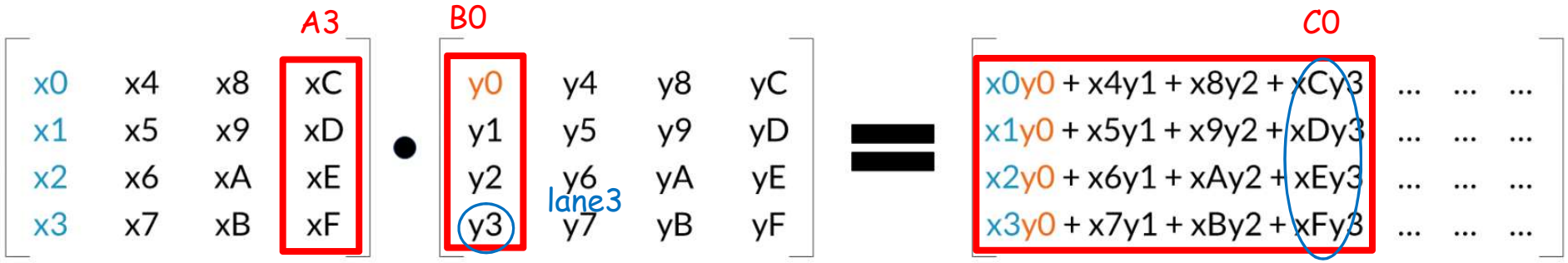


- Elementwise multiplication, followed by elementwise addition (accumulation)
- multiply-accumulate instructions
- e.g., vmlaq_laneq_s32 (a, b, v, lane)

```
 C0 = vmlaq_laneq_s32(C0, A_0, B_0, 0);
 C0 = vmlaq_laneq_s32(C0, A_1, B_0, 1);
 C0 = vmlaq_laneq_s32(C0, A_2, B_0, 2);
 C0 = vmlaq_laneq_s32(C0, A_3, B_0, 3);
...
```

METHOD 2: MULTIPLY-ACCUMULATE

- We can alter the algorithm to avoid the reduction

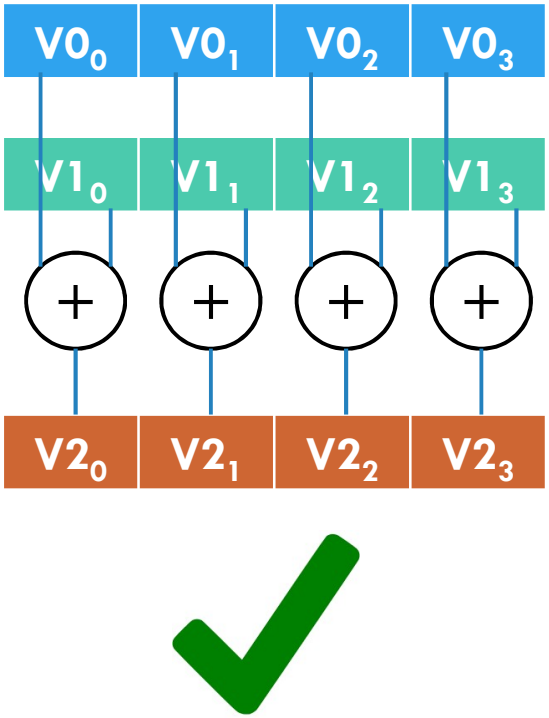


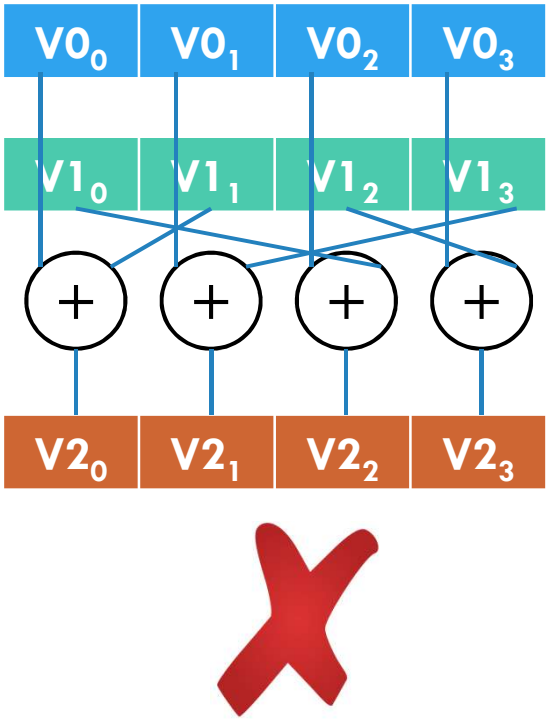
- Elementwise multiplication, followed by elementwise addition (accumulation)
- multiply-accumulate instructions
- e.g., vmlaq_laneq_s32 (a, b, v, lane)

```
 C0 = vmlaq_laneq_s32(C0, A_0, B_0, 0);
 C0 = vmlaq_laneq_s32(C0, A_1, B_0, 1);
 C0 = vmlaq_laneq_s32(C0, A_2, B_0, 2);
 C0 = vmlaq_laneq_s32(C0, A_3, B_0, 3);
...
```

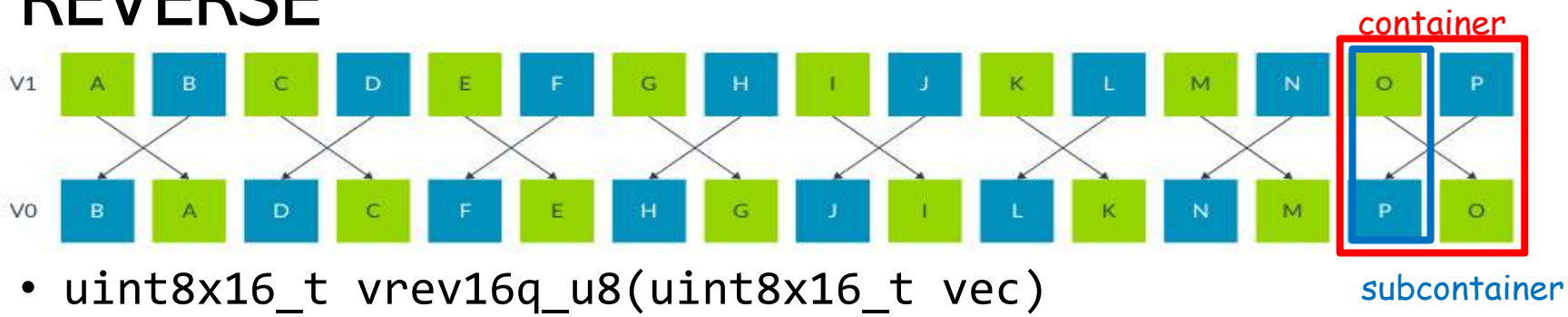
PERMUTATION

- Having data at the right position is very important
- The best way: devise an efficient memory layout in the first place
- Second-best way: use strided load/stores
- Third-best way: use permutation instructions

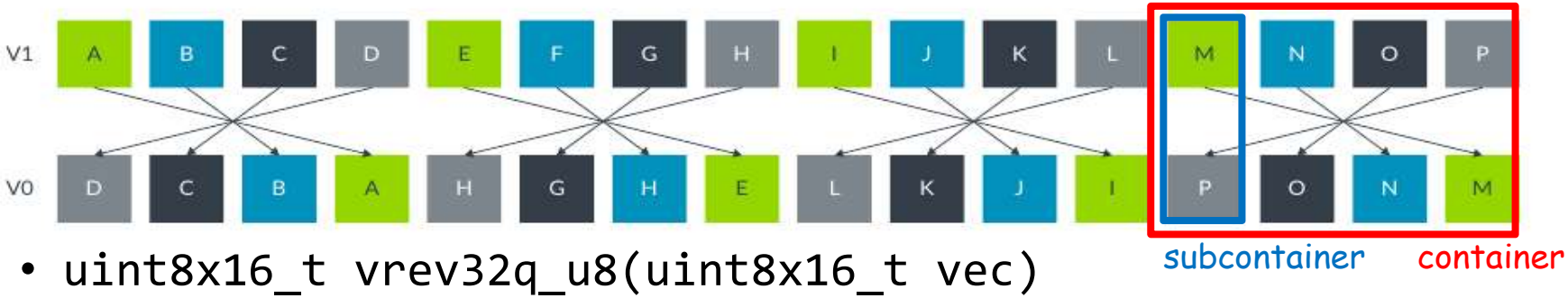




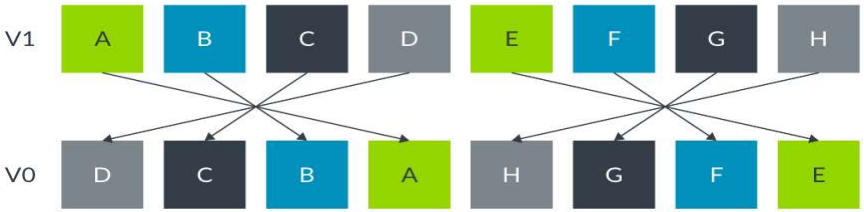
REVERSE



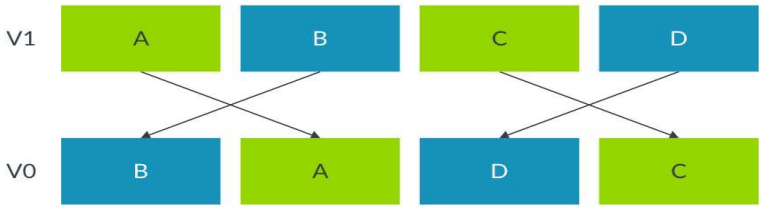
- `uint8x16_t vrev16q_u8(uint8x16_t vec)`
  - vector-reverse, Q register, 16b container, 8b subcontainer



- `uint8x16_t vrev32q_u8(uint8x16_t vec)`
  - vector-reverse, Q register, 32b container, 8b subcontainer



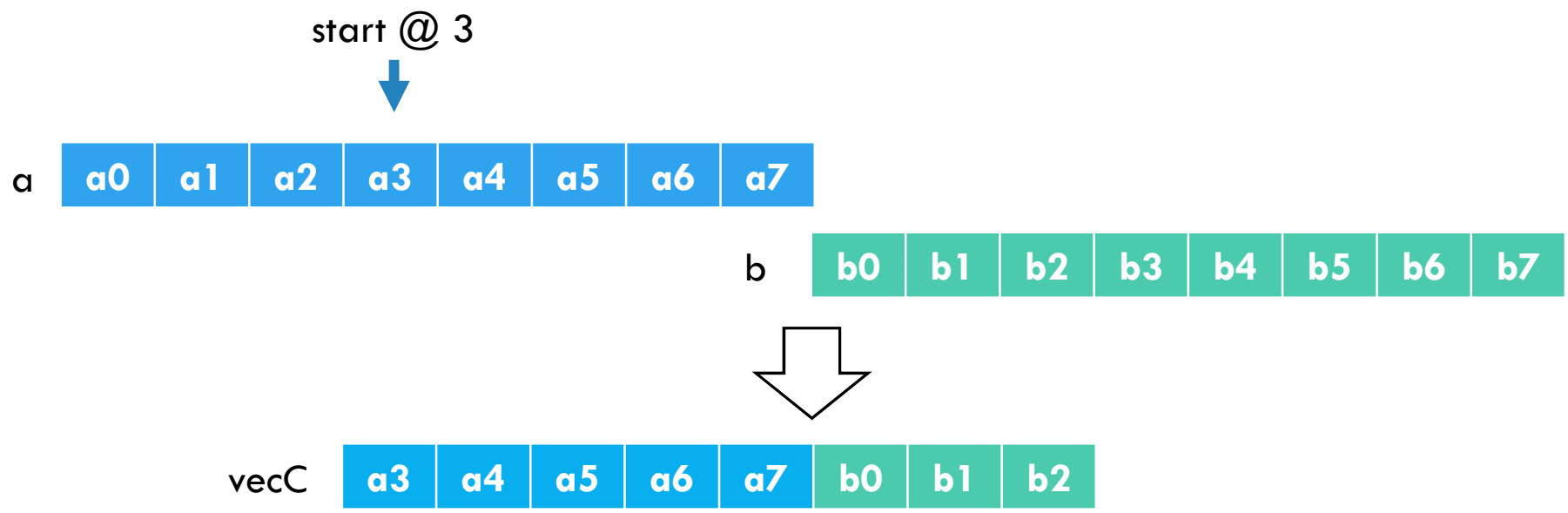
`vrev64q_u16`



`vrev64q_u32`

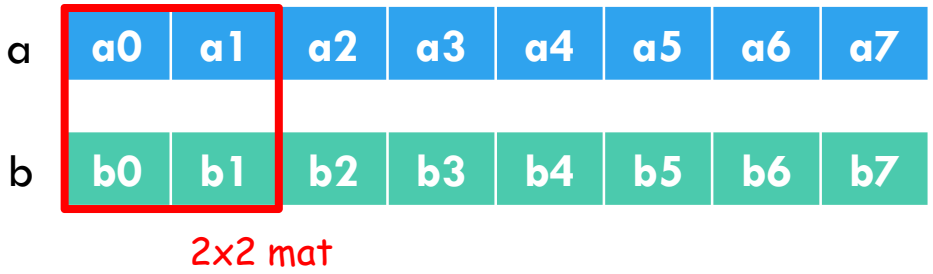
EXTRACT

- Extract from two vectors into one
- `uint16x8_t vextq_u16(uint16x8_t a, uint16x8_t b, const int n)`
  - `e.g., vecC = vextq_u16(a, b, 3)`

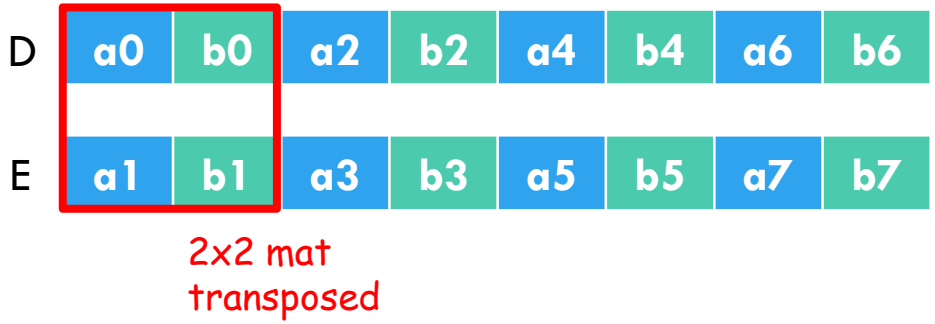


TRANSPOSE

- trn1 interleaves even-numbered elements, trn2 interleaves odd-numbered elements

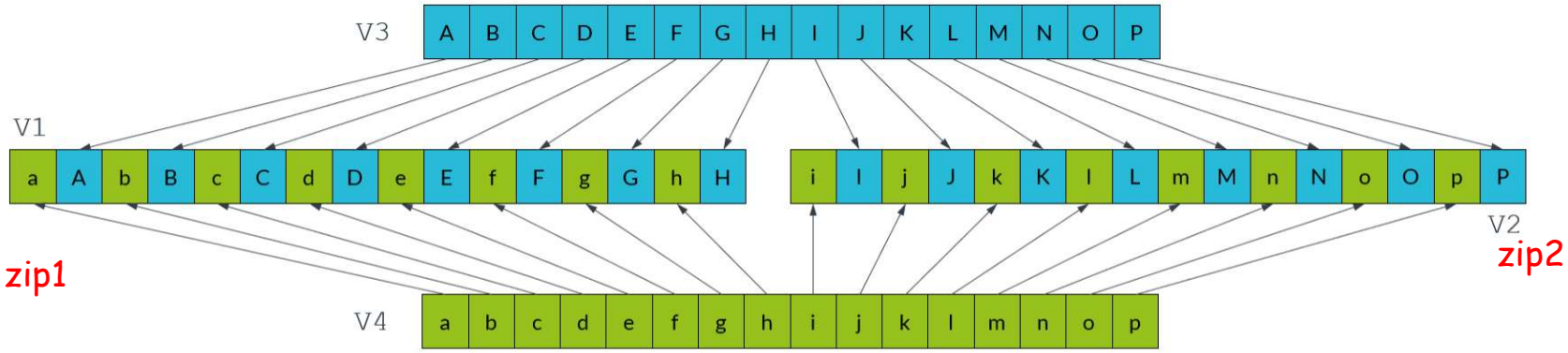


- `uint16x8_t vecD = vtrn1q_u16(vecA, vecB);`
- `uint16x8_t vecE = vtrn2q_u16(vecA, vecB);`

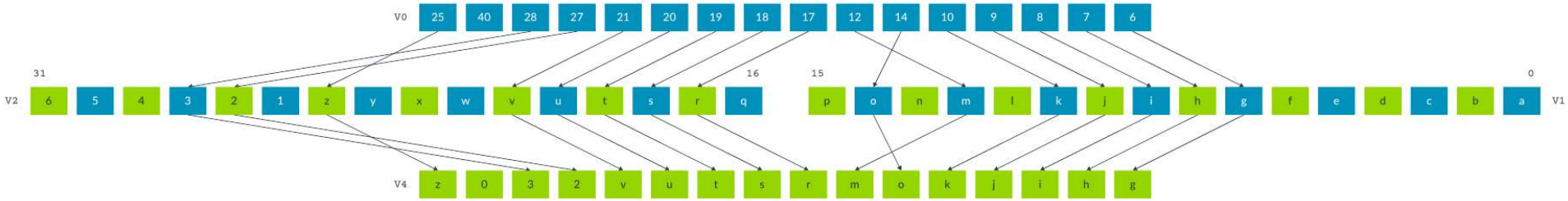


ZIP, TBL

- use ZIP1 and ZIP2 for interleaving of registers
- `uint8x16_t vzip1q_u8( uint8x16_t a, uint8x16_t b)`



- Use TBL and TBX for table lookups
- `uint8x16_t vqtbl2q_u8( uint8x16x2_t t, uint8x16_t idx)`



SUMMARY

- Neon supports parallel data processing in SIMD
- Intrinsics provide a good trade-off point between performance and programming effort
- Final note: Using intrinsics does not guarantee speedup.
  - Only helpful when your compute-bounded
  - …and you make a good code
  - Always measure

- Further reading
  - Optimizing C code with Neon intrinsics - https://developer.arm.com/documentation/102467/0201
  - List of intrinsics - https://arm-software.github.io/acle/neon_intrinsics/advsimd.html
  - NEON™ Programmer's Guide - https://developer.arm.com/documentation/den0018/a/?lang=en
  - Coding for Neon (assembly) – https://developer.arm.com/-/media/Arm%20Developer%20Community/PDF/Neon%20Programmers%20Guide/102159_0104_01_CodingForNeon.pdf?revision=a3235e10-ded9-4ac8-a415-badcafdd56a6